

BANCO DE DADOS PARA E-ASTRONOMIA

Consultas com JOIN e Otimização com SQL e Python

Heleno Campos

LIneA: Laboratório Interinstitucional de e-Astronomia

Maio de 2026



- ◉ O curso será gravado
- ◉ Todo o material ficará disponível para consulta posterior
- ◉ Dúvidas podem ser tiradas no Slack (usuários LIneA):
`lineausers.slack.com` | entrar no canal do curso: `#bd-para-eastronomia`
- ◉ Dinâmica do curso:
 - ◈ Aula expositiva / prática
 - ◈ Estudo de caso
 - ◈ Possibilidade de tirar dúvidas no Slack por uma semana

■ Seção 1 · Consultas SQL Avançadas

- ◉ JOINS, subconsultas, CTEs
- ◉ Funções de agregação, GROUP BY, HAVING
- ◉ Window Functions

■ Seção 3 · Integração com Python

- ◉ Queries parametrizadas
- ◉ Pandas + SQL
- ◉ Gráficos e boas práticas

■ Seção 2 · Otimização e Performance

- ◉ Índices e estratégias
- ◉ EXPLAIN QUERY PLAN
- ◉ Boas práticas

■ Seção 4 · Estudo de Caso Aplicado

- ◉ Catálogo de galáxias DES DR2
- ◉ Pipeline SQL + Python
- ◉ Análise científica real

QUEM SOU EU?

Heleno Campos

Pós Doutorado em Computação @ UFF

Doutor em Computação @ UFF

`helenocampos@id.uff.br`

`helenocampos.github.io`



POR QUE SQLITE?

I O que é SQLite

- ⦿ Banco de dados relacional **embarcado**
- ⦿ Não requer servidor nem configuração
- ⦿ Dados armazenados em um único arquivo .db
- ⦿ Suporte ao padrão SQL (ANSI)
- ⦿ Módulo **sqlite3** nativo no Python

I Perfeito para este curso

Banco local, sem instalação de servidor. Foco no SQL, não na infraestrutura.

I SQLite vs PostgreSQL

	SQLite	PostgreSQL
Servidor	não	sim
Multi-user	limitado	sim
Escala	pequena	grande
Instalação	zero	sim
Uso típico	local	produção

I No LIneA (produção)

O portal LIneA aceita comandos **PostgreSQL** para consulta aos catálogos de dados. SQLite aqui simula a mesma lógica SQL em escala menor.

I Seções 1 e 2: DB Browser for SQLite

- ◉ Interface visual para bancos SQLite
- ◉ Executa queries SQL sem código
- ◉ Navega tabelas e visualiza resultados
- ◉ Download gratuito:
`sqlitebrowser.org`

I Seções 3 e 4: Jupyter Notebook

- ◉ Python + sqlite3 + pandas
- ◉ Queries integradas com análise de dados
- ◉ Gráficos com matplotlib
- ◉ Pipelines reproduzíveis

I Progressão ao longo do curso

Exploração visual (DB Browser) → **Automação e análise** (Python + Jupyter)

Seção 1.

Consultas SQL Avançadas

OBJETIVOS DESTA SEÇÃO

Ao final desta seção você será capaz de:

- ◉ Combinar tabelas com **JOINS** (INNER, LEFT, RIGHT, FULL OUTER)
- ◉ Escrever **subconsultas** e **CTEs** para organizar queries complexas
- ◉ Aplicar **funções de agregação** com GROUP BY e HAVING
- ◉ Entender o conceito de **Window Functions**

O BANCO DE DADOS DO CURSO

Usaremos um banco de dados de **biblioteca** como exemplo didático:

livros

id	identificador único
titulo	título do livro
genero	ficção, romance...
ano	ano de publicação

usuarios

id	identificador único
nome	nome do usuário
cidade	cidade de origem

emprestimos

id	identificador único
usuario_id	chave para usuarios
livro_id	chave para livros
data_emp	data do empréstimo
data_dev	data da devolução

Relações

livros.id = empréstimos.livro_id
usuarios.id = empréstimos.usuario_id

REVISÃO: SELECT, WHERE, ORDER BY

■ Livros de ficção após 2010

```
SELECT id, titulo, ano
FROM   livros
WHERE  genero = 'Ficcao'
      AND ano > 2010
ORDER BY ano
LIMIT 5;
```

■ Resultado (exemplo)

id	titulo	ano
12	Duna	2012
34	Fundacao	2015

■ Ordem lógica de execução

- ❶ FROM: qual tabela?
- ❷ WHERE: quais linhas?
- ❸ SELECT: quais colunas?
- ❹ ORDER BY: em que ordem?
- ❺ LIMIT: quantas linhas?

■ Atenção

A **ordem lógica** difere da **sintaxe escrita**!

POR QUE PRECISAMOS DE JOINS?

I O problema

- ◉ livros: título, gênero, ano
- ◉ empréstimos: quem pegou, quando
- ◉ Informação **distribuída por design**, evitando duplicação de dados

I Pergunta típica

“Quais livros de ficção foram emprestados em 2024?”

- ⇒ gênero está em livros
- ⇒ data está em empréstimos
- ⇒ precisamos **cruzar as tabelas**

I Quatro tipos de JOIN

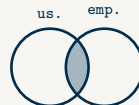
Tipo	Retorna
INNER	só linhas com par em ambas
LEFT	todas da esq. + NULLs
RIGHT	todas da dir. + NULLs
FULL OUTER	todas as linhas de ambas

INNER JOIN: APENAS A INTERSEÇÃO

Usuários que fizeram empréstimos

```
SELECT u.nome, e.data_emp
FROM   usuarios AS u
INNER JOIN emprestimos e
        ON u.id = e.usuario_id
LIMIT 4;
```

O que faz



Retorna apenas linhas com par em ambas as tabelas. Usuários sem empréstimo não aparecem.

Lembrete: JOIN sem tipo equivale a INNER JOIN.

Resultado (exemplo)

nome	data_emp
Ana	2024-03-01
Beto	2024-02-15

LEFT JOIN: PRESERVANDO A TABELA ESQUERDA

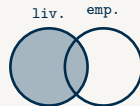
■ Todos os livros, com ou sem empréstimo

```
SELECT l.titulo, e.data_emp
FROM   livros AS l
LEFT JOIN emprestimos e
      ON l.id = e.livro_id
LIMIT 4;
```

■ WHERE e.data_emp IS NULL

Encontra livros nunca emprestados.

■ O que faz



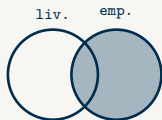
Mantém todos os livros da esquerda. Ausência de match gera NULL.

■ Resultado (exemplo)

titulo	data_emp
Duna	2024-03-01
Dom Quixote	NULL

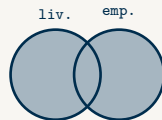
RIGHT JOIN E FULL OUTER JOIN

RIGHT JOIN



Mantém todas as linhas da tabela **direita**. Na prática, prefira reescrever como LEFT JOIN trocando a ordem das tabelas.

FULL OUTER JOIN



Retorna **todas** as linhas de ambas as tabelas. Útil para auditar registros sem correspondência em qualquer lado.

Suporte em SQLite

SQLite **não** suporta FULL OUTER JOIN nativamente. Emule com LEFT JOIN UNION ALL LEFT JOIN com as tabelas trocadas.

FUNÇÕES DE AGREGAÇÃO

I O que são

Recebem um conjunto de valores e retornam um único resultado. Resumem dados de um grupo de linhas em uma única linha de saída.

I Quando usar

- ⦿ Contar o total de registros
- ⦿ Calcular média ou soma de valores
- ⦿ Encontrar o menor ou o maior valor
- ⦿ Resumir grupos de dados

I As principais funções

COUNT(*)	conta todas as linhas
COUNT(col)	ignora NULLs
SUM(col)	soma os valores
AVG(col)	média aritmética
MIN(col)	menor valor
MAX(col)	maior valor

I COUNT(*) vs COUNT(col)

COUNT(*) inclui NULLs.

COUNT(col) ignora NULLs.

FUNÇÕES DE AGREGAÇÃO: EXEMPLO

■ Estatísticas dos livros de ficção

```
SELECT COUNT(*) AS total,  
       MIN(ano) AS mais_antigo,  
       MAX(ano) AS mais_novo  
FROM   livros  
WHERE  genero = 'Ficcao';
```

■ Resultado (exemplo)

total	mais_antigo	mais_novo
142	1965	2023

■ O que retorna

Uma única linha com estatísticas de todos os livros de ficção. GROUP BY não é necessário aqui: queremos um único resumo de todo o conjunto.

GROUP BY: AGRUPANDO RESULTADOS

I O que é

Agrupar linhas com o mesmo valor em uma coluna. Cada grupo vira uma única linha no resultado, onde funções de agregação calculam estatísticas desse grupo.

I Quando usar

- ⦿ Contar itens por categoria
- ⦿ Calcular médias ou somas por grupo
- ⦿ Resumir dados por período ou região

I Estrutura

```
SELECT coluna,  
        FUNCAO(col2)  
FROM    tabela  
GROUP BY coluna;
```

I Regra importante

No SELECT, toda coluna que não for uma agregação deve aparecer no GROUP BY.

GROUP BY: EXEMPLO

■ Livros por gênero

```
SELECT genero,
       COUNT(*) AS total,
       MIN(ano) AS mais_antigo
FROM   livros
GROUP  BY genero
ORDER  BY total DESC;
```

■ Resultado (exemplo)

genero	total	mais_antigo
Ficcao	142	1965
Romance	89	1812

■ Execução

- ❶ FROM: carrega livros
- ❷ GROUP BY: forma um grupo por cada valor de genero
- ❸ SELECT: calcula COUNT e MIN dentro de cada grupo
- ❹ ORDER BY: ordena o resultado

HAVING: FILTRANDO GRUPOS

I O que é

HAVING filtra grupos **após** a agregação. Funciona como um WHERE, mas aplicado ao resultado do GROUP BY.

I Quando usar

- ◉ Quando o critério envolve uma função de agregação
- ◉ Exemplos: gêneros com mais de 10 livros; usuários com mais de 5 empréstimos em aberto

WHERE vs HAVING

	WHERE	HAVING
Filtra	linhas	grupos
Momento	antes GROUP BY	após GROUP BY
Usa agreg.?	não	sim

HAVING: EXEMPLO

■ Gêneros com mais de 10 livros

```
SELECT genero,
       COUNT(*) AS total
FROM   livros
GROUP  BY genero
HAVING COUNT(*) > 10
ORDER  BY total DESC;
```

■ Resultado (exemplo)

genero	total
Ficcao	142
Romance	89

■ Por que não WHERE?

WHERE COUNT(*) > 10 seria inválido:

WHERE é executado antes do GROUP BY e não conhece ainda o resultado de COUNT(*).

HAVING é executado depois, quando os grupos já existem.

SUBCONSULTAS EM WHERE

Livros publicados após a média

```
SELECT id, titulo, ano
FROM   livros
WHERE  ano > (
        SELECT AVG(ano)
        FROM   livros
      ) LIMIT 4;
```

Resultado (exemplo)

id	titulo	ano
34	Fundacao	2015
58	Duna	2019

Como funciona

- 1 Subconsulta calcula $AVG(ano)$
 ≈ 2008
- 2 Query externa usa esse valor como limite
- 3 Retorna livros com
 $ano > 2008$

Tipos de subconsulta

Escalar: retorna 1 valor ($> AVG(...)$)

IN: retorna um conjunto ($WHERE id IN (...)$)

SUBCONSULTA IN VS INNER JOIN

Com IN

```
SELECT titulo, genero
FROM livros
WHERE id IN (
    SELECT livro_id
    FROM emprestimos
    WHERE data_dev
        IS NULL
);
```

Quando preferir

- ⦿ Filtro simples e legível
- ⦿ Subconsulta independente

Com INNER JOIN

```
SELECT DISTINCT
    l.titulo, l.genero
FROM livros AS l
JOIN emprestimos e
    ON l.id = e.livro_id
WHERE e.data_dev
    IS NULL;
```

Quando preferir

- ⦿ Precisa de colunas de ambas as tabelas
- ⦿ Geralmente mais eficiente

SUBCONSULTAS CORRELACIONADAS

■ Livros emprestados 3 ou mais vezes

```
SELECT l.id, l.titulo
FROM   livros AS l
WHERE (
    SELECT COUNT(*)
    FROM   emprestimos
    WHERE  livro_id = l.id
) >= 3;
```

■ Resultado (exemplo)

id	titulo
12	Duna
34	Fundacao

■ Como funciona

Para **cada linha** de livros, a subconsulta é re-executada usando o `l.id` daquela linha. Por isso é chamada de **correlacionada**: depende de um valor da query externa.

■ Cuidado com performance

Executa **uma vez por linha**. Com tabelas grandes, prefira JOIN ou CTE.

I CTEs: *Common Table Expressions*

```
WITH recentes AS (  
    SELECT id, titulo, genero  
    FROM    livros  
    WHERE   ano >= 2020  
)  
SELECT genero, COUNT(*) AS total  
FROM    recentes  
GROUP   BY genero;
```

I Resultado (exemplo)

genero	total
Ficcao	8
Romance	5

I O que são CTEs?

Common Table Expressions: subqueries nomeadas, definidas antes da query principal com WITH nome AS (...).

I Por que usar?

- ⊙ **Legibilidade:** nome descreve o propósito
- ⊙ **Reuso:** referenciável várias vezes na query
- ⊙ **Depuração:** cada CTE pode ser testada

WINDOW FUNCTIONS: VISÃO GERAL

I O que são

Calculam valores por linha sem colapsar os resultados, diferente do GROUP BY que resume cada grupo em uma única linha.

Sintaxe: `FN() OVER (PARTITION BY col ORDER BY col)`

PARTITION BY divide as linhas em grupos, mas **mantém todas as linhas** individuais no resultado.

I Exemplos de uso

- ⦿ Ranking dos livros mais emprestados dentro de cada gênero
- ⦿ Acumulado de empréstimos mês a mês
- ⦿ Comparar os empréstimos de cada usuário com a média do grupo

I Nesta aula

Window Functions não serão aprofundadas. Consulte a documentação do SQLite para mais detalhes.

SEÇÃO 1: O QUE APRENDEMOS

I Técnicas dominadas

- ◉ **JOINS:** INNER, LEFT, RIGHT, FULL OUTER
- ◉ **Agregação:** COUNT, SUM, AVG, MIN, MAX
- ◉ **GROUP BY:** resumir dados por grupo
- ◉ **HAVING:** filtrar grupos pós-agregação
- ◉ **Subconsultas:** escalar, IN, correlacionada
- ◉ **CTEs:** legibilidade com WITH
- ◉ **Window Functions:** introdução e casos de uso

I Próxima seção

Seção 2: Otimização e Performance

- ◉ Por que queries ficam lentas?
- ◉ Índices: o que são e como ajudam
- ◉ EXPLAIN QUERY PLAN
- ◉ Boas práticas de escrita SQL

Seção 2.

Otimização e Performance

I Ao final desta seção você saberá

- ◉ Identificar queries com varredura desnecessária
- ◉ Criar índices simples e compostos
- ◉ Ler e interpretar EXPLAIN QUERY PLAN
- ◉ Aplicar boas práticas de escrita SQL

I Motivação

Com 500 000 empréstimos e 100 000 livros, uma query mal escrita leva **segundos**; otimizada, executa em **milissegundos**.

O PROBLEMA: VARREDURA COMPLETA DA TABELA

I Query sem índice

```
-- SQLite examina TODAS as linhas
-- (100 000 livros) para achar
-- as que passam no filtro
SELECT id, titulo, genero
FROM   livros
WHERE  genero = 'Ficcao'
      AND ano > 2000;
```

I O que acontece internamente

- ⦿ SQLite lê cada linha do disco
- ⦿ Testa a condição WHERE
- ⦿ Descarta as que não passam
- ⦿ Repete para as 100 000 linhas

I Consequência

Retornando 1 000 livros, o banco leu **100×** mais do necessário.

CRIANDO ÍNDICES COM CREATE INDEX

I Sintaxe e exemplos

```
-- Índice simples (uma coluna)
CREATE INDEX idx_genero
  ON livros(genero);

-- Índice em coluna de JOIN
CREATE INDEX idx_emp_livro
  ON emprestimos(livro_id);

-- Remover índice
DROP INDEX IF EXISTS idx_genero;
```

Estrutura B-tree separada que ordena os valores da coluna para busca binária.

I Quando criar

- ⦿ Colunas em WHERE frequentes
- ⦿ Colunas em JOIN ON
- ⦿ Colunas em ORDER BY e GROUP BY

I Custo vs benefício

- ⦿ Leitura: muito mais rápida
- ⦿ Escrita: levemente mais lenta
- ⦿ Disco: espaço adicional usado

COMO FUNCIONA O ÍNDICE B-TREE?

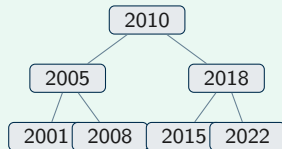
I Como funciona

- ◉ Sem índice: ler todas as 100 000 linhas
- ◉ Com B-tree: navegar pela árvore ordenada
- ◉ Busca binária: $\log_2(100\,000) \approx 17$ comparações até chegar ao valor
- ◉ Analogia: índice de livro, vai direto ao assunto

I Regra prática

Crie índices em colunas com muitos valores distintos e usadas frequentemente em WHERE, JOIN ON, ORDER BY.

I Estrutura B-tree (simplificada)



Cada nó: valor de ano
+ ponteiro para a linha da tabela.

EXPLAIN QUERY PLAN: SCAN VS SEARCH

Antes e depois do índice

```
-- SEM índice (idx_genero inexistente)
EXPLAIN QUERY PLAN
SELECT id FROM livros
WHERE genero = 'Ficcao';
-- QUERY PLAN
-- `--SCAN livros

-- COM índice (após CREATE INDEX)
EXPLAIN QUERY PLAN
SELECT id FROM livros
WHERE genero = 'Ficcao';
-- QUERY PLAN
-- `--SEARCH livros USING
--     INDEX idx_genero (genero=?)
```

Como interpretar

- ⦿ **SCAN**: percorre todas as linhas
- ⦿ **SEARCH**: acessa via B-tree
- ⦿ **USING INDEX**: mostra qual índice
- ⦿ Prefira SEARCH em filtros frequentes

ÍNDICES COMPOSTOS: MÚLTIPLOS FILTROS

Índice em duas colunas

```
-- Índice composto
CREATE INDEX idx_gen_ano
  ON livros(genero, ano);

-- Esta query USA o índice:
SELECT id, titulo
FROM   livros
WHERE  genero = 'Ficcao'
      AND ano > 2010;

-- Esta NÃO usa idx_gen_ano:
-- WHERE ano > 2010
-- (coluna genero ausente)
```

Regra da ordem

- ⊙ (A, B): funciona com A ou A+B
- ⊙ Somente B: ignora o índice
- ⊙ Coluna mais seletiva primeiro
- ⊙ Verifique com EXPLAIN QUERY PLAN

BOAS PRÁTICAS DE ESCRITA SQL

I Prefira...

- ◉ `SELECT col1, col2` em vez de `SELECT *`
- ◉ Filtros sobre colunas **indexadas**
- ◉ `JOIN` em vez de subconsulta correlacionada
- ◉ CTEs para legibilidade em queries complexas
- ◉ `LIMIT` ao explorar dados desconhecidos

I Evite...

- ◉ Funções sobre colunas indexadas no `WHERE` (impedem o uso do índice)
- ◉ `SELECT *` em produção
- ◉ Subconsultas correlacionadas em tabelas grandes
- ◉ `OR` entre colunas (prefira `IN`)

BOAS PRÁTICAS: FUNÇÕES EM COLUNAS INDEXADAS

Evite: desabilita o índice

```
-- Funcao na coluna:  
-- nao usa idx_gen_ano  
SELECT id FROM livros  
WHERE UPPER(genero)  
      = 'FICCAO';  
-- => SCAN: 100000 linhas
```

Prefira: usa o índice

```
-- Coluna isolada:  
-- usa idx_gen_ano  
SELECT id FROM livros  
WHERE genero = 'FICCAO';  
-- => SEARCH: ~17 passos
```

Regra geral

Caso não queira perder o uso do índice, não aplique função à coluna indexada no WHERE.

I Evite: múltiplos OR

```
SELECT id, titulo
FROM   livros
WHERE  genero = 'Ficcao'
      OR genero = 'Romance'
      OR genero = 'Historia';
```

Três predicados separados:
mais difícil de ler e otimizar.

I Prefira: IN

```
SELECT id, titulo
FROM   livros
WHERE  genero IN (
        'Ficcao',
        'Romance',
        'Historia');
```

Um único predicado:
mais legível e melhor otimizado.

I Outros usos de IN

- ⊙ WHERE genero NOT IN ('Infantil') — excluir conjunto
- ⊙ WHERE id IN (SELECT ...) — subconsulta como filtro

SEÇÃO 2: O QUE APRENDEMOS

I Técnicas de otimização

- ◉ **Full table scan**: o problema de base
- ◉ **B-tree**: estrutura dos índices
- ◉ **CREATE INDEX**: solução principal
- ◉ **EXPLAIN QUERY PLAN**: diagnóstico
- ◉ **Índices compostos**: filtros múltiplos
- ◉ **Boas práticas**: funções, OR vs IN

I Próxima seção

Seção 3: Integração com Python

- ◉ Conectar Python ao SQLite
- ◉ Queries parametrizadas
- ◉ Pandas + SQL

Seção 3.

Integração com Python

OBJETIVOS DA SEÇÃO

I Ao final desta seção você saberá

- ◉ Conectar Python a bancos SQLite
- ◉ Usar queries parametrizadas com segurança
- ◉ Carregar resultados SQL em DataFrames Pandas
- ◉ Gerar gráficos a partir de dados do banco
- ◉ Construir pipelines reproduzíveis

I Ferramentas

- ◉ **sqlite3**: módulo nativo do Python
- ◉ **pandas**: análise de dados
- ◉ **matplotlib**: visualização
- ◉ **pd.read_sql_query**: integração direta

Conexão, cursor e busca

```
import sqlite3

# Conectar ao banco de dados
conn = sqlite3.connect(
    'dados/biblioteca.db')
cursor = conn.cursor()

# Executar e buscar resultado
cursor.execute(
    "SELECT COUNT(*) FROM livros")
total = cursor.fetchone()[0]
print(f"Livros: {total}")

conn.close()
```

Módulo sqlite3

- ⦿ **conn**: objeto de conexão
- ⦿ **cursor**: executa queries
- ⦿ **fetchone()**: 1 resultado
- ⦿ **fetchall()**: todos os resultados
- ⦿ **conn.close()**: liberar recursos

QUERIES PARAMETRIZADAS: SEGURANÇA

Placeholder ? evita injeção SQL

```
# EVITE: f-string abre injeção SQL
# WHERE genero = '{genero}'

# CORRETO: placeholder ?
genero, ano_min = 'Ficcao', 2000

cursor.execute("""
    SELECT id, titulo, ano
    FROM    livros
    WHERE   genero = ?
           AND ano    > ?
""", (genero, ano_min))

rows = cursor.fetchall()
```

Por que usar ?

- ◉ SQLite escapa os valores automaticamente
- ◉ Impede injeção SQL
- ◉ Funciona para qualquer tipo de dado
- ◉ Valores passados como tupla

Nunca faça

f"WHERE genero = '{g}'" permite que o usuário quebre a query.

SQL direto para DataFrame

```
import sqlite3
import pandas as pd

with sqlite3.connect(
    'dados/biblioteca.db'
) as conn:
    df = pd.read_sql_query("""
        SELECT id, titulo,
               genero, ano
        FROM    livros
        WHERE   ano > 2000
        """, conn)

print(df.shape)    # (N, 4)
print(df.dtypes)
```

Vantagens

- ⦿ Retorna **DataFrame** pronto para análise
- ⦿ Tipos inferidos automaticamente
- ⦿ Integra com numpy e matplotlib
- ⦿ **with**: fecha conexão automaticamente

Filtrar, agrupar, estatísticas

```
# Filtrar no DataFrame
mask = df['ano'] >= 2010
recentes = df[mask]
print(f"{len(recentes)} livros")

# Estatísticas por gênero
res = (
    df.groupby('genero')
      ['ano']
      .agg(['mean', 'count'])
)
print(res)
```

#	genero	mean	count
#	Ficcao	2005.3	142
#	Romance	1998.7	89

Operações comuns

- ◉ **df[mask]**: filtragem booleana
- ◉ **groupby**: agrupamento
- ◉ **agg**: múltiplas estatísticas
- ◉ **describe()**: resumo completo
- ◉ **plot()**: gráficos rápidos

GERANDO GRÁFICOS A PARTIR DO BANCO

I Livros por gênero com matplotlib

```
with sqlite3.connect('dados/biblioteca.db') as conn:
    df = pd.read_sql_query("""
        SELECT genero, COUNT(*) AS total
        FROM livros GROUP BY genero
        ORDER BY total DESC
        """, conn)
```

```
fig, ax = plt.subplots(figsize=(6,4))
ax.bar(df['genero'], df['total'])
ax.set_xlabel('Genero')
ax.set_ylabel('Total de livros')
plt.savefig('livros_genero.png')
```

I Fluxo

- 1 Consulta SQL agrega os dados
- 2 DataFrame carregado com resultado
- 3 matplotlib plota diretamente das colunas do DataFrame
- 4 Figura salva em arquivo

I Boas práticas

Faça a agregação no **banco**, não no Python. Carregue apenas o resumo, não todos os dados.

BOAS PRÁTICAS: BANCO DE DADOS GRANDE

I Problemas comuns

- ⦿ Carregar tabela inteira no Python e filtrar depois (memória!)
- ⦿ `SELECT *` traz colunas desnecessárias
- ⦿ Laços Python linha a linha em vez de operações SQL
- ⦿ Conexões não fechadas

I Regra de ouro

Filtre, agregue e selecione colunas no SQL, antes de trazer os dados para o Python.

I Boas práticas

- ⦿ Use `WHERE` e `LIMIT` na query
- ⦿ Selecione só as colunas necessárias
- ⦿ Faça `JOINS` e agregações no banco
- ⦿ Use **with** para fechar conexões
- ⦿ Explore com `LIMIT 100` antes de carregar o conjunto completo
- ⦿ Prefira **índices** nas colunas de filtro (Seção 2)

PIPELINE COMPLETO: SQL + PYTHON

I Do banco ao CSV em poucas linhas

```
import sqlite3, pandas as pd

DB = 'dados/biblioteca.db'

with sqlite3.connect(DB) as conn:
    df = pd.read_sql_query("""
        SELECT u.cidade,
               COUNT(*) AS empréstimos
        FROM    usuarios AS u
        JOIN    empréstimos e
              ON u.id = e.usuario_id
        GROUP  BY u.cidade
        ORDER  BY empréstimos DESC
        """, conn)

df.to_csv('result.csv', index=False)
print(f"{len(df)} cidades exportadas")
```

I Fluxo reproduzível

- 1 Definir query SQL
- 2 Conectar com **with**
- 3 Carregar em DataFrame
- 4 Processar / visualizar
- 5 Exportar resultados

I Dica

Salve a query em uma variável separada para facilitar a leitura e a manutenção do código.

SEÇÃO 3: O QUE APRENDEMOS

I Técnicas dominadas

- ◉ **sqlite3**: conexão nativa Python + SQLite
- ◉ **Placeholders**: queries seguras com ?
- ◉ **fetchone/fetchall**: buscar resultados
- ◉ **read_sql_query**: SQL para DataFrame
- ◉ **matplotlib**: gráficos a partir do banco
- ◉ **Boas práticas**: filtrar no banco, não no Python

I Próxima seção

Seção 4: Estudo de Caso Aplicado

- ◉ Catálogo de galáxias DES DR2
- ◉ Tabelas objects e observations
- ◉ Pipeline SQL + Python completo

Seção 4.

Estudo de Caso Aplicado

O BANCO DE DADOS REAL: DES DR2

■ objects (150 000 linhas)

id	identificador único
ra, dec	coordenadas em graus
color_gr	cor $g - r$ pré-calculada
color_ri	cor $r - i$ pré-calculada
extended_class	tipo (3 = galáxia)

■ object_bands (674 451 linhas)

band_id	identificador único
object_id	chave para objects
band	banda (g, r, i, z, y)
num_obs	número de exposições
mag_auto	magnitude nessa banda
magerr_auto	erro da magnitude

■ Relação

`objects.id = object_bands.object_id`
1 : N

O QUE VAMOS CONSTRUIR

I Pipeline de análise fotométrica

- 1 Selecionar galáxias do DES DR2
- 2 Filtrar por brilho e cobertura multi-banda
- 3 Gerar diagrama cor-magnitude
- 4 Rankear e exportar catálogo de alvos

I Técnicas da aula aplicadas

- ◉ **CTE + JOIN** entre as duas tabelas
- ◉ **EXPLAIN QUERY PLAN** para diagnóstico
- ◉ **pandas + matplotlib** para análise
- ◉ Pipeline reproduzível em Jupyter

ETAPA 1: SELEÇÃO DE GALÁXIAS

Galáxias com cobertura multi-banda

```
WITH cobertura AS (  
    SELECT object_id  
    FROM    object_bands  
    GROUP BY object_id  
    HAVING COUNT(DISTINCT band) >= 3  
)  
SELECT o.id, o.ra, o.dec,  
       ob_i.mag_auto      AS mag_auto_i,  
       o.color_gr  
FROM   objects o  
JOIN   cobertura c ON o.id = c.object_id  
JOIN   object_bands ob_i  
       ON o.id = ob_i.object_id  
WHERE  ob_i.mag_auto      < 23.0  
       AND o.extended_class = 3  
       AND ob_i.band = 'i';
```

Filtros aplicados

- ⦿ `mag_auto < 23` (banda i): limite de brilho
- ⦿ `extended_class = 3`: galáxias
- ⦿ `bands >= 3`: cobertura multi-banda
- ⦿ `band = 'i'`: banda i

Resultado

≈ 107 000 galáxias
prontas para análise.

ETAPA 2: CARREGANDO COM PYTHON

SQL para DataFrame

```
import sqlite3, pandas as pd

DB = 'dados/processed/course_database.db'
with sqlite3.connect(DB) as conn:
    df = pd.read_sql_query(SQL, conn)

print(df.shape)      # (107048, 5)
print(df.dtypes)
# id                  int64
# ra                  float64
# dec                 float64
# mag_auto_i          float64
# color_gr            float64
```

Inspeção do DataFrame

- ⦿ **df.shape**: (linhas, colunas)
- ⦿ **df.dtypes**: tipos inferidos
- ⦿ **df.describe()**: estatísticas
- ⦿ **df.isnull().sum()**: nulos

ETAPA 3: DIAGRAMA COR-MAGNITUDE

■ visualização

```
fig, ax = plt.subplots(figsize=(6,4))
ax.scatter(
    df['mag_auto_i'],
    df['color_gr'],
    s=1, alpha=0.3,
    c='steelblue')

ax.set_xlabel('Magnitude i')
ax.set_ylabel('Cor g-r')
ax.invert_xaxis()
ax.set_title('DES DR2 - CMD')
plt.savefig('cmd_des_dr2.png', dpi=150)
```

■ Diagrama cor-magnitude

- ⊙ Eixo X: **magnitude i** (brilho)
- ⊙ Eixo Y: **cor $g - r$** (temperatura)
- ⊙ `invert_xaxis()`: mag menores são mais brilhantes

ETAPA 4: RANKING E EXPORTAÇÃO

Ranquear por brilho e salvar

```
# Rank global por brilho (mag menor)
df['rank'] = df['mag_auto_i'].rank(
    method='min')

# Top 500 objetos mais brilhantes
top = df[df['rank'] <= 500]

# Exportar catálogo
cols = ['id', 'ra', 'dec',
        'mag_auto_i', 'color_gr']
top[cols].to_csv(
    'alvos_des_dr2.csv', index=False)
print(f"Exportados: {len(top)}")
# Exportados: 500
```

Resultado

- ◉ **rank()**: posição global por brilho
- ◉ **method='min'**: empates recebem menor rank
- ◉ Saída: 500 galáxias mais brilhantes
- ◉ CSV pronto para observação

Arquivo gerado

```
alvos_des_dr2.csv
id, ra, dec,
mag_auto_i, color_gr
```

CONCLUSÃO DO CURSO

I Jornada percorrida

- ◉ **Seção 1:** JOINs, agregação, CTEs, Window Functions
- ◉ **Seção 2:** Índices, B-tree, EXPLAIN QUERY PLAN, boas práticas
- ◉ **Seção 3:** Python + SQL, gráficos, pipelines reproduzíveis
- ◉ **Seção 4:** Pipeline com dados reais do **DES DR2**

I Próximos passos

- ◉ Explorar o portal LIneA com o DES DR2 completo
- ◉ PostgreSQL para bases de produção
- ◉ SQLAlchemy para acesso em Python
- ◉ astropy + coordenadas para cruzamento de catálogos

Obrigado!

Materiais e portal: linea.org.br

REFERÊNCIAS E LEITURAS COMPLEMENTARES

I Documentação oficial

- ⦿ **SQLite**
sqlite.org/docs.html
- ⦿ **Python sqlite3**
docs.python.org/3/library/sqlite3
- ⦿ **pandas**
pandas.pydata.org/docs
- ⦿ **matplotlib**
matplotlib.org

I Tutoriais SQL interativos

- ⦿ SQLZoo.net
- ⦿ [W3Schools SQL](https://W3Schools.com)

I Livros recomendados

- ⦿ *Learning SQL* de Alan Beaulieu (O'Reilly)
- ⦿ *SQL for Data Analysis* de Cathy Tanimura (O'Reilly)

I LIneA e DES

- ⦿ Portal LIneA:
linea.org.br
- ⦿ LIneA User Query:
userquery.linea.org.br
- ⦿ Catálogos de dados do LIneA:
data.linea.org.br/